# Discussion 5

TCP & IP

# Outline

- Assignment 2 & Midterm Logistics
- TCP
- IP

# Logistics

# Assignment 2

- Assignment 2 is due February 28
  - This is **after** the midterm
- Please get started now!
  - Assignment 2 is a significantly harder project than Assignment 1
  - If you don't start now, you risk both not being able to finish the assignment, and not having enough time to study for the midterm
- Autograder for Assignment 2 will be released soon™
- Use different classes to keep state for different parts of your proxy!
  - Clean and well-thought out design is important for this project

# Midterm



- The midterm will be on **February 21 from 7:00 to 8:30 PM**
- Additional practice exams released from last semester on course GitHub
- Room assignments by uniqname:
  - AA - DB: Chrysler 133
  - DC - ZZ: Chrysler 220
- If you have SSD accomadations, watch out for more information

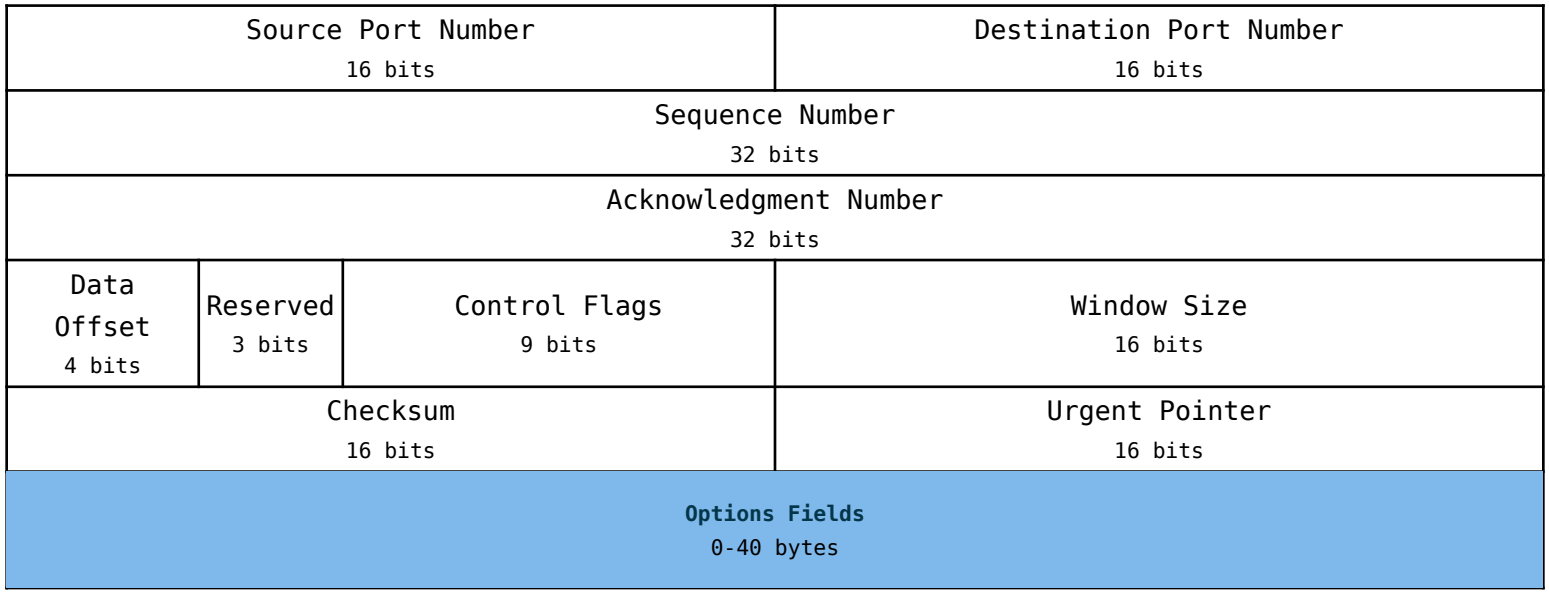
# Midterm

- Midterm covers:
  - Discussions 1-6
  - Project 1
  - All lectures through “More Congestion Control”
- Allowed materials:
  - One double-sided 8.5" $\times$ 11" note sheet
  - Basic calculator (optional, and not needed)
- Keep an eye on Ed for more information!

# TCP

# TCP: Overview

- TCP is a transport layer protocol on top of IP.
  - It provides a **reliable**, **ordered**, and **error-checked byte stream abstraction**
- TCP headers look like the following:



# TCP: Overview



- **Reliable**
  - TCP attempts to resend lost packets; provides a “lossless” view to the application
- **Ordered**
  - TCP guarantees that the receiver application reads the data in the same order that it was sent
- **Error-checked**
  - TCP provides a checksum to detect errors in the received data
- **Byte stream**
  - TCP provides the view of a “stream of bytes” rather than discrete packets to the application

# TCP: Congestion Control



- Additionally, TCP provides congestion control
  - TCP will try to ensure that the sender does not overwhelm the receiver or the network

# TCP: Congestion Control



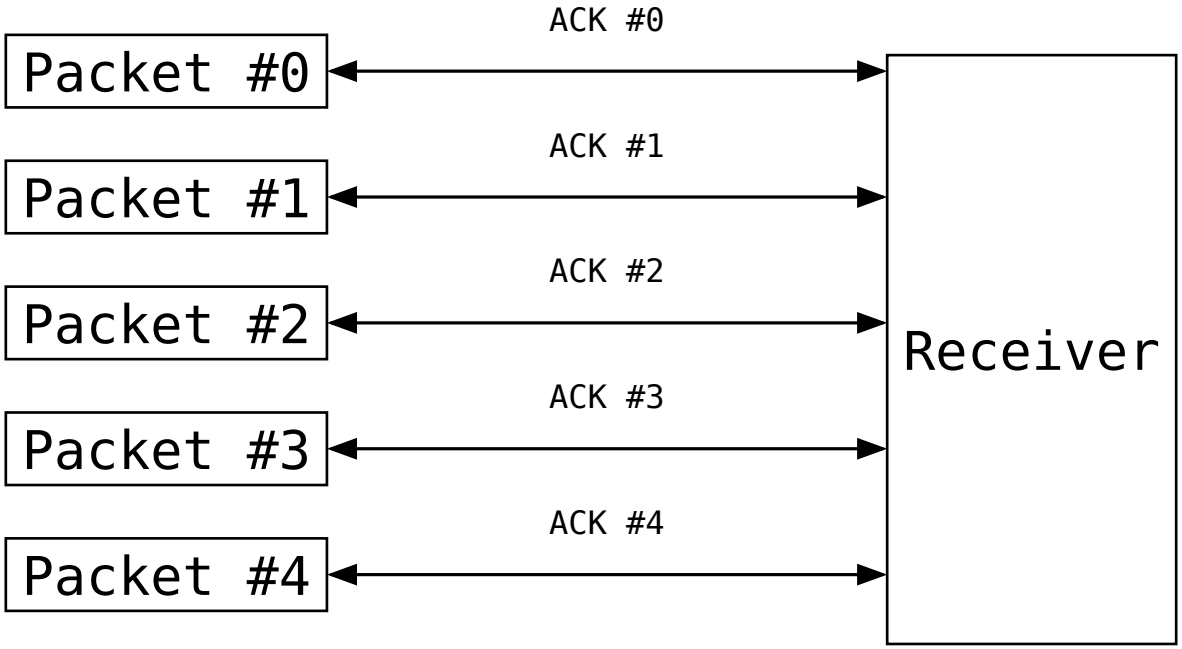
- Congestion control state maintained at the sender
  - **Congestion Window (`CWND`)**
    - A window that determines how much data the sender can send at once
    - `CWND` is adjusted based on network conditions
  - **Slow Start Threshold (`ssthresh`)**
    - A threshold that determines when the update rate for the congestion window slows down
  - **Timer**
    - A sender-side timer to detect packet loss when no `ACK` is received
  - **Duplicate `ACK` Count**
    - How many duplicate `ACK`s have been received
    - This indicates a one-off packet drop

# TCP: Congestion Control



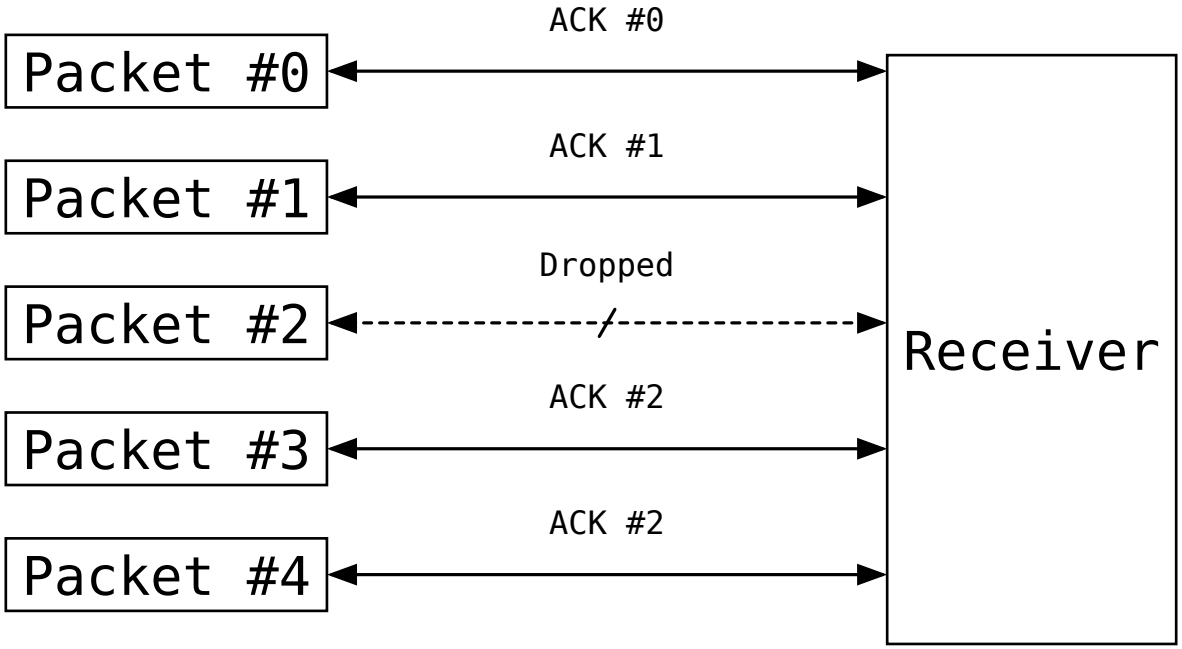
- Before diving in deeper, it's important note what some different events look like in TCP

# TCP: Successful Transmission of Window

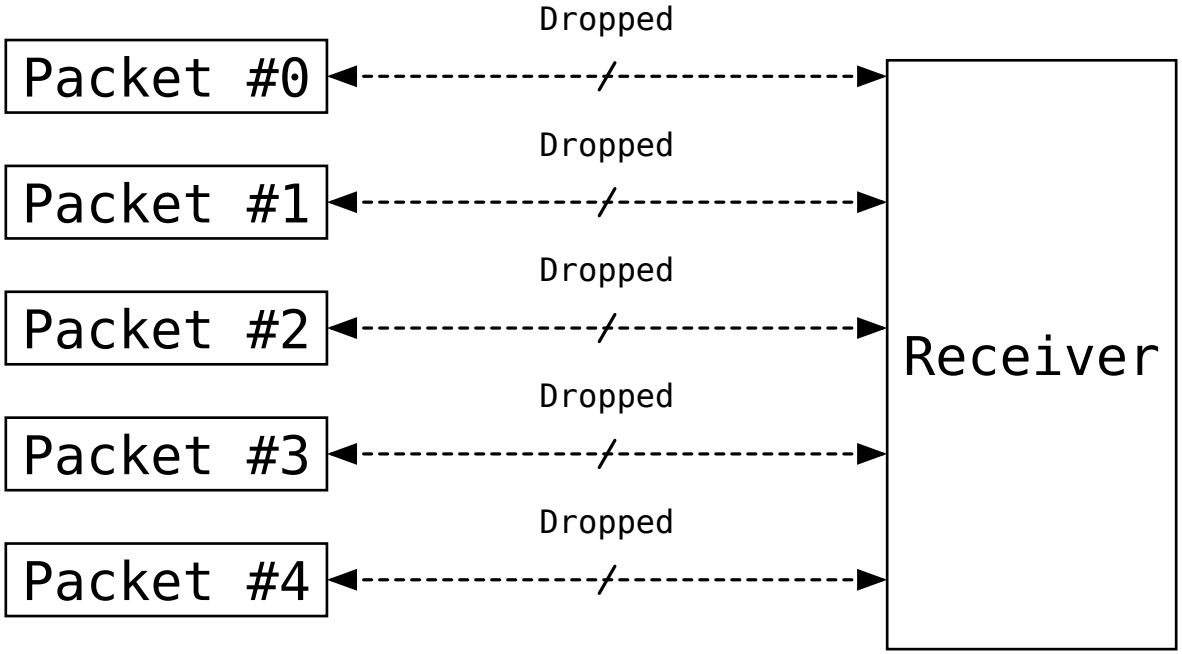


# TCP: One-Off Drop





# TCP: Timeout




Sender doesn't hear anything back!

# TCP: Congestion Control



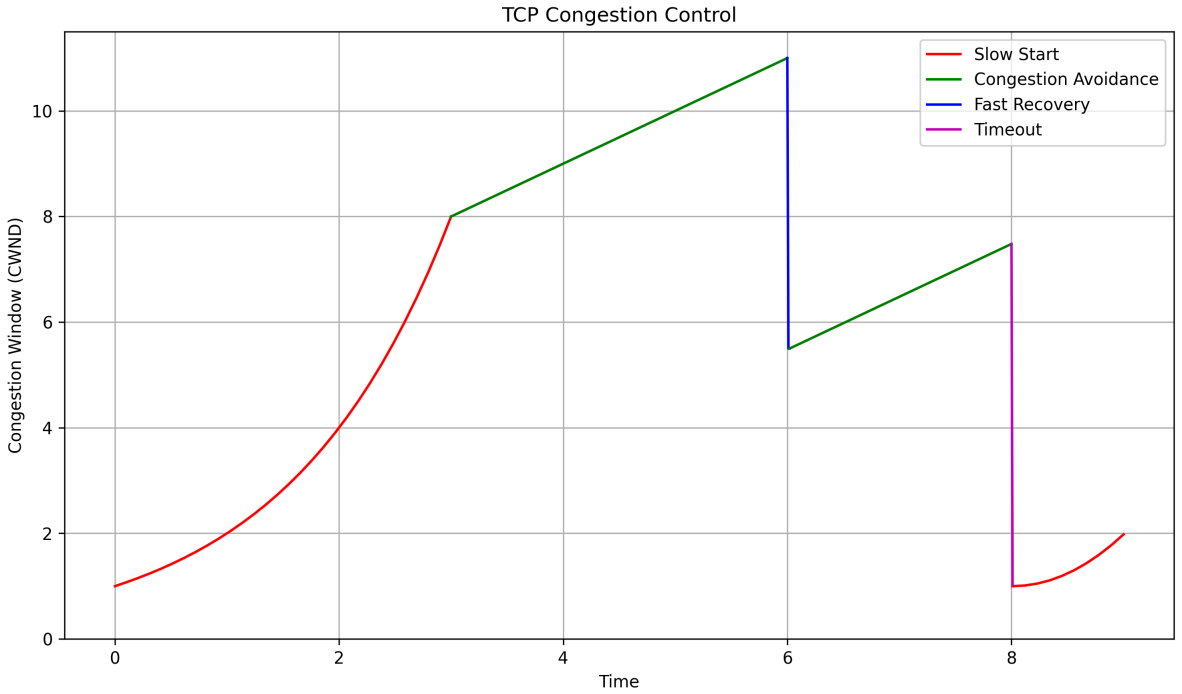
- Events that update the sender state:
  - `ACK` for new data
    - Good! Increase the congestion window
  - `Timeout` on the sender-side
    - Really bad! Reset the congestion window and decrease the slow start threshold
  - `Duplicate ACK`
    - Not that bad. Retransmit the missing packet and decrease the congestion window slightly

# TCP: Example

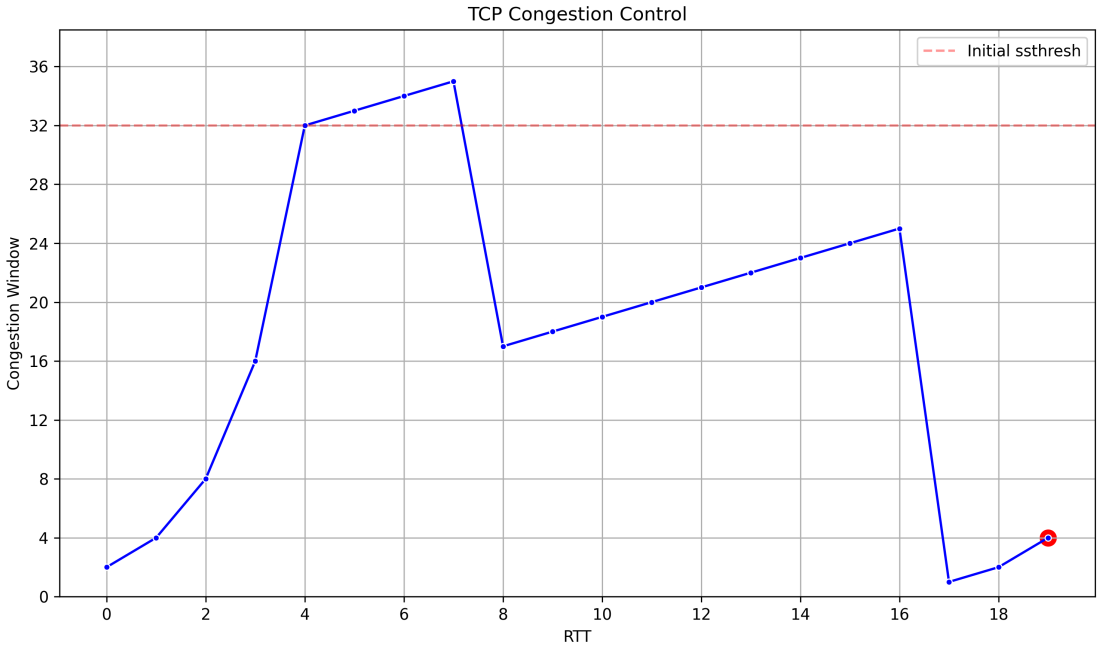


**Example**

In the below graph, let's identify the different phases of TCP congestion control



# TCP: Example 2




**Answer**

We cut the congestion window in half when we get a duplicate `ACK`, but `ssthresh` is only adjusted to $\frac{\texttt{CWND}}{2} = 16$ after the timeout at the end.

# IP

# IP: Overview

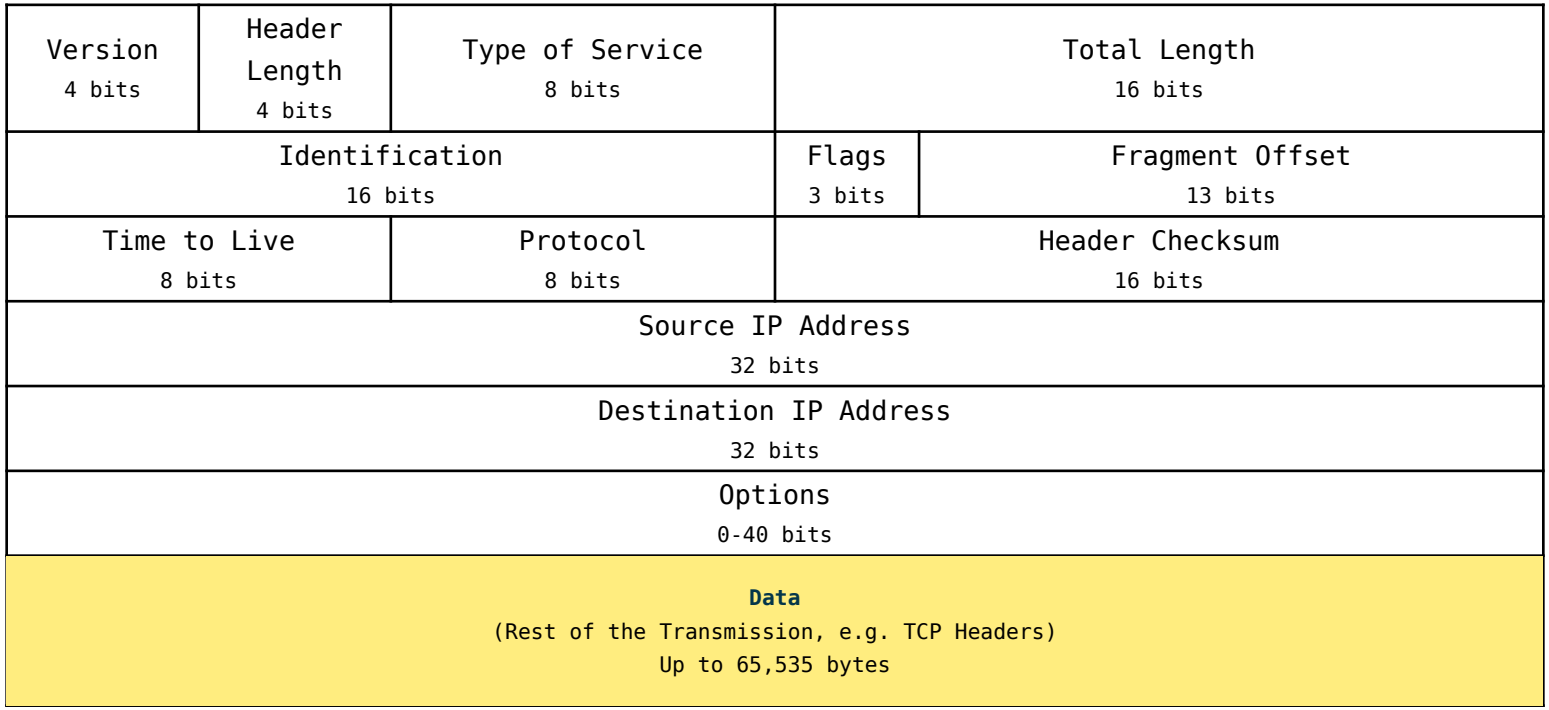


- Hosts on the internet need some way to route packets to each other
  - We need the internet-equivalent of an address
- IP provides a connectionless, best-effort delivery service
  - No guarantees about delivery, order, or error checking
  - Allows packets to be routed from source to destination via one or more routers

# IP: Protocol



- The IPv4 header looks like the following:



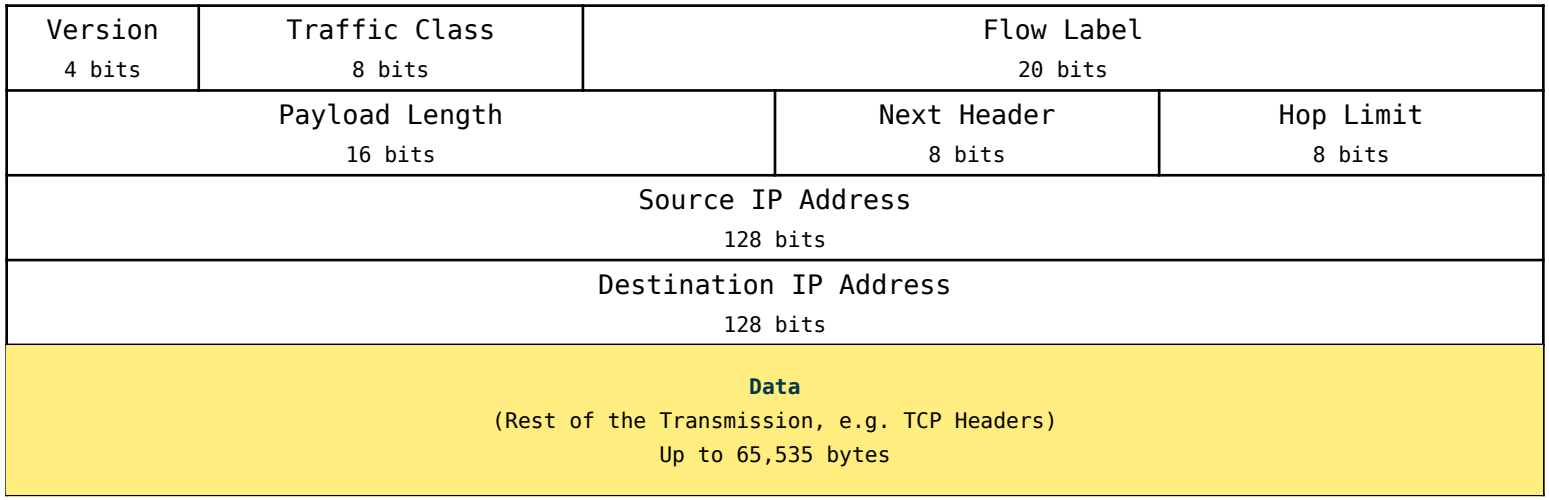
# IP: IPv6



- IPv4 only has $2^{32}$ addresses
  - Too few for the number of devices on the internet
- IPv6 is a new standard that makes the following changes:
  - IP addreses are now 128 bits, allowing for $2^{\{128\}}$ addresses
  - Fixed-length header
  - IP no longer provides features like checksum, fragmentation, and explicit options
  - Time-to-live is now replaced with a hop limit
  - The header is now 40 bytes, fixed

# IP: IPv6

- IPv6 addresses look like the following:



# IP: Routing



- IP routing is done using a **routing table**
  - A table that maps destination IP addresses to the next hop router
  - Uses **longest prefix matching** to determine the next hop

# IP Routing: Example



**Example**

Given the following routing table, determine the next hop for the destination IP address `192.168.1.100`

| Destination IP Address | Subnet Mask | Subnet Mask (Hex) | Next Hop Router |
|---|---|---|---|
| `10.0.0.0` | `255.0.0.0` | `FF 00 00 00` | `R1` |
| `192.160.0.0` | `255.240.0.0` | `FF F0 00 00` | `R2` |
| `192.168.0.0` | `255.255.0.0` | `FF FF 00 00` | `R3` |
| `0.0.0.0` | `0.0.0.0` | `00 00 00 00` | `R4` |

**Answer**

The destination IP address matches both `192.160.0.0` and `192.168.0.0` with their subnet masks. However, `192.168.0.0` is a more specific match, so `R3` will be chosen as the next hop.

# IP Fragmentation: Example



**Example**

Suppose that a host is sending a 4000 byte IP packet (including the IP header) to a destination with an MTU of 1500 bytes. You can assume that an IP header is 20 bytes.

For each fragment in the IP packet, describe the size of the fragment (including its IP header) and the fragment offset.

**Answer**

If our entire packet is 4000B, and the size of an IP header is 20B, the size of the data is $4000 - 20 = 3980$B.

Then, for the first fragment, we can fit 1480B of data plus the IP header for that fragment. For this fragment, the fragment offset is 0.

For the second fragment, we can fit the 1480B of data plus the IP header for that fragment. For this fragment, the fragment offset is $\frac{1480}{8} = 185$.

Finally, the last fragment will have the remaining 1020B of data plus the IP header for that fragment. For this fragment, the fragment offset is $\frac{1480+1480}{8} = 370$.